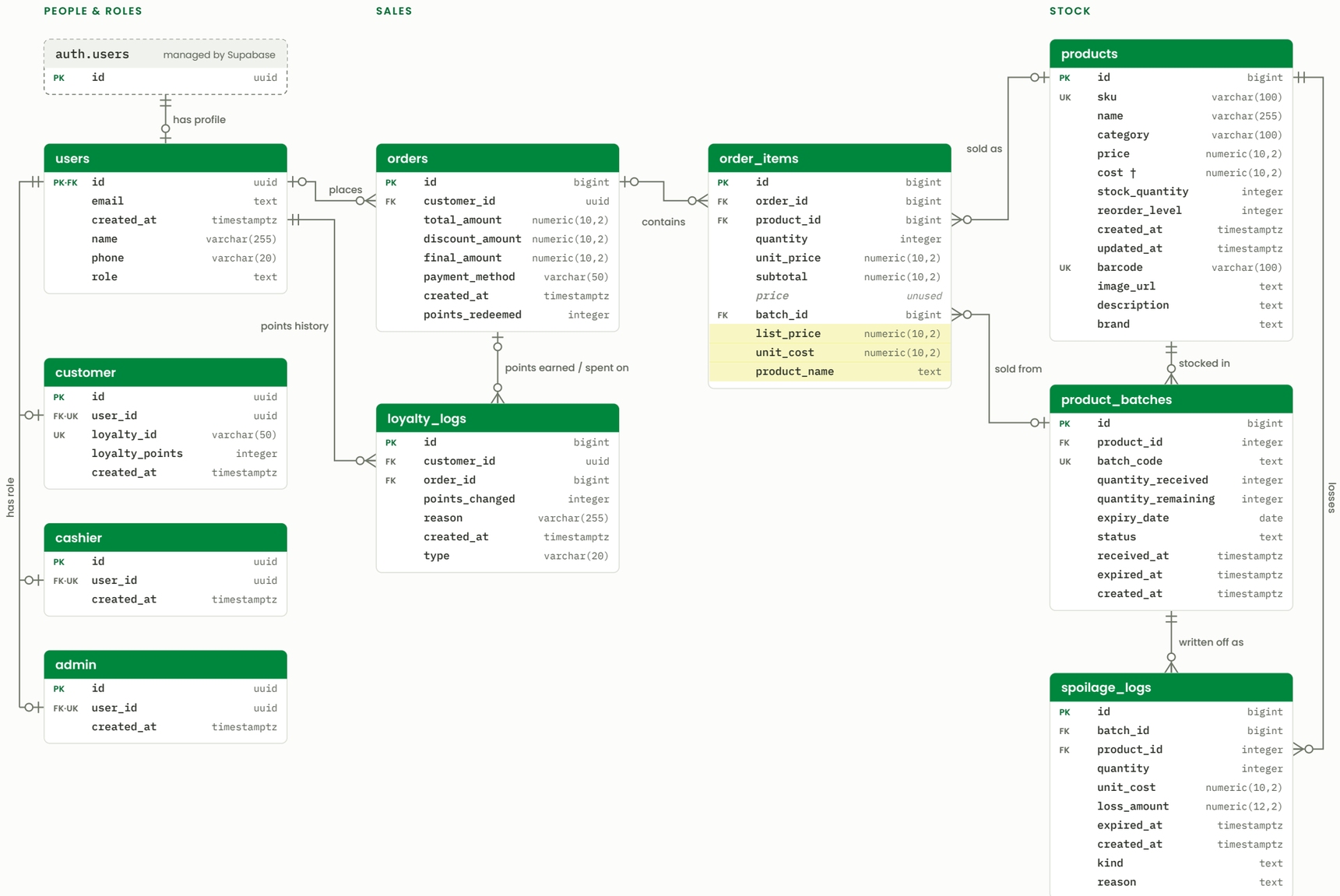
SUPABASE · PUBLIC SCHEMA · LIVE DATABASE, 2 NOV 2026

SuperMarket Express database

Ten tables, plus Supabase's own `auth.users`, read from the live project. Each line joins a foreign key (FK) to the row it points at, in crow's-foot notation: the symbol at each end says how many rows can be on that side.

exactly one · zero or one · zero or many · PK primary key · FK foreign key · UK unique · copied at the moment of sale · † readable by admins only



People on the left, sales in the middle, stock on the right. Every sale (orders) belongs to a user account and lists its lines (order_items); each line points at the product and the batch its units came from. Stock arrives in product_batches and leaves either through a sale or as a write-off in spoilage_logs.

Relationships

From → to	Through	Cardinality	If the parent is deleted
auth.users → users	`users.id`	`1 : 0..1`	profile deleted too
users → customer	`customer.user_id`	`1 : 0..1`	deleted too
users → cashier	`cashier.user_id`	`1 : 0..1`	deleted too
users → admin	`admin.user_id`	`1 : 0..1`	deleted too
users → orders	`orders.customer_id`	`0..1 : 0..*`	order kept, member cleared
users → loyalty_logs	`loyalty_logs.customer_id`	`1 : 0..*`	deleted too
orders → order_items	`order_items.order_id`	`0..1 : 0..*`	deleted too
orders → loyalty_logs	`loyalty_logs.order_id`	`0..1 : 0..*`	log kept, order cleared
products → order_items	`order_items.product_id`	`0..1 : 0..*`	blocked while sold
product_batches → order_items	`order_items.batch_id`	`0..1 : 0..*`	blocked while sold
products → product_batches	`product_batches.product_id`	`1 : 0..*`	blocked
product_batches → spoilage_logs	`spoilage_logs.batch_id`	`1 : 0..*`	blocked
products → spoilage_logs	`spoilage_logs.product_id`	`1 : 0..*`	blocked

Rules the database enforces

Table	Rule
`users`	`role` is admin, cashier or customer (the highest role held); `phone` is a PH mobile number used by at most one member (checked by `set_member_phone`)
`customer`	`loyalty_points` never below 0; `loyalty_id` unique, given automatically (LOY-000001, …)
`loyalty_logs`	`type` is EARNED or REDEEMED
`products`	`sku` and `barcode` unique; barcode not blank; `description` at most 300 characters; `brand` not blank, at most 60 characters
`product_batches`	`batch_code` unique; received > 0; 0 ≤ remaining ≤ received; `status` is active, expired, spoiled or manual
`spoilage_logs`	`quantity` > 0; cost and loss never negative; `kind` is expired, damaged or spoiled; a reason is required
`orders`	`payment_method` is Cash, Card or Loyalty Points (checked by the checkout function rather than a table rule)

Reading notes

- **Accounts and roles.** `users` is the app's copy of each Supabase sign-in. Everyone gets a `customer` row (their loyalty account); a row in `cashier` or `admin` adds that role.
- **Sales point at the account.** `orders.customer_id` and `loyalty_logs.customer_id` hold `users.id`, not `customer.id`.
- **Frozen sale lines.** A trigger copies `list_price`, `unit_cost` and `product_name` into each `order_items` row when it's saved, so editing a product later doesn't change past orders or profit. `unit_price` is what was actually charged (15% less for near-expiry units).
- **Private cost.** Only admins can read `products.cost`; customers and cashiers see the price only.
- **Stock movement.** A sale lowers `products.stock_quantity` and each batch's `quantity_remaining`, soonest expiry first. Expired, damaged or spoiled units are recorded in `spoilage_logs` with their cost as the loss.
- **Unused column.** `order_items.price` is left over from an early version; checkout never fills it, so it's always 0.